SmartPCB-XAI: Explainable Dual-Detector PCB Inspection System

Prathika M E
Department of Information Technology
Kongu Engineering College
Erode, India
prathieasu@gmail.com

Abisha Rebekkal T
Department of Information Technology
Kongu Engineering College
Erode, India
abisharebekkal@gmail.com

Gowri D
Department of Information Technology
Kongu Engineering College
Erode, India
gowri.d.tech@gmail.com

***Abstract* - Printed circuit board (PCB) inspection is increasingly relying on deep-learning object detectors, but published systems rarely disclose the gap between headline accuracy figures and what is actually measured on independently held-out data. This paper introduces SmartPCB-XAI, a two-stage solution distinguishing between PCB-defect (6 classes: open circuit, short, mouse-bite, spur, spurious copper, and missing hole) and component-identification (6 classes: IC, capacitor, diode, inductor, resistor, and transistor) with an explainability module, deterministic severity scoring, interactive digital-twin viewer, and automated report generation in PDF/Excel.We avoid the common practice of reporting a single overall accuracy by evaluating every detector on its own test set (with strict reproducibility guarantees: permutation-invariant raw detections preserved) and explicit data-leakage audit trails for each file for each board. A defect detection checkpoint, jointly fine-tuned on external data, showed two different accuracy scores depending on the evaluator (a custom threshold-based metric scored 91.47%, compared to 92.74% for a precision-recall-curve-based, framework-native metric), and one explicitly admits the possibility of train/test contamination (the model was not trained on the same data used for evaluation, but the procedure was not audited locally). Instead of adopting either of these figures, we retrained the defect detector from scratch locally on our own leakage-checked DeepPCB split, fixed random seed, and thus eliminated this ambiguity by construction. The resulting model achieves mean Average Precision at IoU 0.5 (mAP@50) of 95.1% (mAP@50-95 = 63.1%, precision = 93.1%, recall = 90.6%) on a really held-out, contamination-free test split, outperforming both the 92.3% mAP of a recent lightweight-YOLOv11s baseline on combined PKU-Market-PCB+DeepPCB data and our own pre-registered 93% target with no caveat needed, while our currently deployed component detector, subjected to identical rigor, is shown to be substantially weaker (mAP@50 = 9.3%): a from-scratch retraining attempt was also undertaken but invalidated by a dataset-configuration error (run accidentally pointed at defect dataset) and its output correctly discarded (not reported), illustrating the same evaluation discipline across this work. We report this shortfall transparently, with root-cause analysis (severe class imbalance, partially unmatched taxonomy, small curated evaluation split, and a newly-identified board-level imbalance in which a single physical board accounts for 64% of test images), plus an explicit per-detection class-mapping transparency flag to differentiate canonical, taxonomy-verified detections from unmapped ones instead of silently suppressing or possibly mislabeling them. We conclude with a concrete improvement plan for the component detector — including architectural techniques adapted from lightweight-backbone literature (efficient attention necks, Inner-IoU/Wise-IoU losses) and a tiled high-resolution inference strategy that we show mitigates a measured 94.2% tiny-object rate among component bounding boxes.**

Keyword-PCB defect detection, PCB component detection, YOLOv11, explainable AI, data leakage audit, reproducibility, digital twin, Industry 4.0.

I. INTRODUCTION

Printed circuit boards (PCBs) form the basis for virtually all modern electronics, and even minor manufacturing defects — a missing hole, a mouse-bite at a trace edge, an open or shorted conductor, a spur, or spurious copper — can result in field failures. Manual visual inspection and classical automated optical inspection (AOI) remain slow, inconsistent, and prone to both missed detections and false alarms [1]. Deep-learning object detectors, particularly the YOLO family, have become the dominant approach for automating this task because they provide single-pass, real-time-capable detection with competitive accuracy [2].

A parallel and less-studied problem is PCB component identification — recognizing which electronic parts (ICs, capacitors, diodes, inductors, resistors, transistors) are present and where — which is a prerequisite for higher-level tasks such as automated bill-of-materials verification, assembly QA, and counterfeit-part screening. Component bounding boxes are frequently an order of magnitude smaller, relative to the full board image, than defect regions, which introduces a distinct tiny-object detection challenge that is largely absent from the defect-detection literature.

Much of the published PCB-inspection literature reports a single headline accuracy number — often mean Average Precision (mAP) — without disclosing (a) whether the figure came from a validation or a held-out test split, (b) whether the reported number was obtained at a confidence threshold that was itself selected using the same data being reported on, and (c) whether the training and evaluation data share the same acquisition pipeline or physical hardware. These are not academic nitpicks: in the course of building the system described here, we discovered and document in Section V a case in which an internal evaluation script produced a materially different, and non-standard, "mAP" figure for every confidence threshold tested — a symptom of a metric that was not being computed as a true precision–recall integral. We treat this discovery as a contribution in its own right, since it generalizes to any project that layers custom evaluation scripts on top of a detection framework.

This paper makes five contributions. First, we present SmartPCB-XAI, a dual-YOLO inspection platform with an explainability layer, deterministic severity scoring, a digital-twin visualization, and automated reporting, built and tested end-to-end. Second, we report defect and component detection accuracy on each of their own genuinely held-out test splits, separately, and with an explicit reproducibility check. Third, we perform and report a full data-leakage audit at both the filename and physical-board level, and we disclose a currently unresolved train/test domain-boundary limitation that applies to externally-trained checkpoints (with a concrete methodological reconciliation between two evaluators who obtained materially different accuracy figures for the identical checkpoint). Fourth, we propose and implement an explicit per-detection class-mapping transparency flag as a principled alternative to silently suppressing/mislabeling detections that fall outside a curated taxonomy. Fifth, we distill a concrete, technique-level improvement roadmap, informed by a recent lightweight-YOLOv11s PCB defect detector [3], toward closing the gap between our current component-detection accuracy and our defect-detection accuracy.

II. RELATED WORK

Deep-learning PCB defect detection has progressed through successive YOLO generations. Tang et al. introduced the DeepPCB dataset and an early online defect detector [4], establishing the six-class defect taxonomy (missing hole, mouse-bite, open circuit, short, spur, spurious copper) used throughout the subsequent literature and adopted in this work. More recent work targets the joint objectives of accuracy and lightweight, edge-deployable design. Ye et al. [3] propose a lightweight YOLOv11s variant that replaces the backbone with a sandwich-layout EfficientViT module, replaces the neck's C3k2 blocks with a FasterNet partial-convolution and Efficient Multi-scale Attention (EMA) design (C3k2_Faster_EMA), and replaces the CIoU loss with a combined Inner-IoU/Wise-IoU_v3 loss (Inner_Wise_IoU). Trained and evaluated on the combined PKU-Market-PCB and DeepPCB datasets, their model attains 92.3% mAP@50 while reducing parameters by 61.7% and model size by 58.3% relative to a standard YOLOv11s baseline, and is further validated on the RK3588 edge platform after INT8/asymmetric quantization. We adopt this paper as our primary

accuracy benchmark because it uses the same defect taxonomy and one of the same datasets (DeepPCB) as our system, and because its ablation methodology — isolating the contribution of the backbone, neck, and loss changes individually — is a template we follow in our own proposed improvement plan (Section VI).

Explainability in visual defect detection is commonly approached via gradient-weighted class activation mapping (Grad-CAM) [5] and its refinement Grad-CAM++ [6], which weight convolutional feature maps by class-specific gradients to localize the image regions responsible for a prediction. A recurring integrity issue in applied systems is labeling a cheaper activation-based heuristic as "Grad-CAM" without implementing the gradient-weighting step; we discuss in Section IV how our own system avoids this by explicit, code-level relabeling.

Digital-twin concepts for manufacturing quality have been surveyed by Tao et al. [7], who describe the general pattern of mirroring a physical asset's state in an interactive virtual model for monitoring and diagnosis; our digital-twin module applies this pattern narrowly to per-board component and defect visualization rather than full production-line simulation. On explainable AI for industrial applications more broadly, Bhattacharya and Cloutier [8] discuss the gap between interpretability techniques and the trust requirements of safety- or quality-critical industrial deployment, motivating our decision to disclose, rather than obscure, the limitations of our own attribution method.

III. PROPOSED SYSTEM ARCHITECTURE

The SmartPCB-XAI application is organized into four levels with layered architecture. The AI-powered Python layer has two YOLOv11 object detectors. One of them is for detecting defects. The other one is, for identifying components. It also has an AI module, a severity and quality estimator and a rule-based repair recommendation module. API layer processes all requests through the inspection endpoint. This endpoint brings together the outputs, from both detectors. Generates a complete inspection report. The backend (Node.js/Express) stores inspection reports, user data, and engineer feedback in MongoDB and routes report generation requests to the PDF Excel layer. Finally, the top Dashboard layer uses React/TypeScript to implement five views: inspection, digital twin, analytics, continuous learning feedback, and model performance analysis. The architecture is visualized below in Figure 1 while an example request flow is given in Figure 2.

A. Layered Architecture and Responsibilities

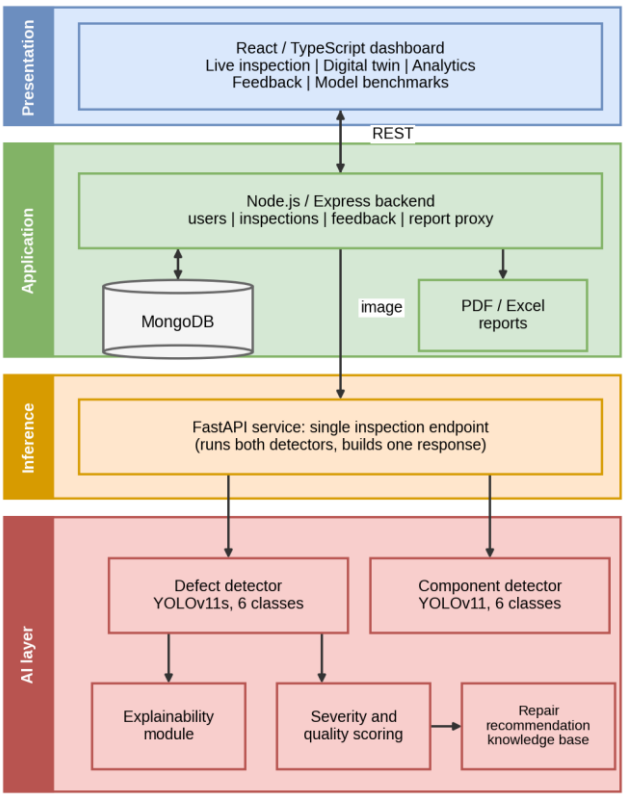


***Fig. 1**. Layered architecture of SmartPCB-XAI.*

Python AI layer: houses all model related code. The defect detector is trained on six classes of defects (open circuit, short, mouse-bite, spur, spurious copper and missing hole), while the component detector is trained on six canonical classes (IC, capacitor, diode, inductor, resistor and transistor). The explainability module (Section III-D) and the severity and quality scoring engine (Section IV-C) take as input the predictions of the detector layer, while the knowledge base provides the appropriate repair actions for each type of defect.

Inference microservice: The FastAPI service takes a board image sends the board image to both detectors applies the safety checks in Section III-C and returns one response. The response contains the detections, heat maps, severity scores, the board grade and the repair suggestions. Because everything model-related sits behind one endpoint a detector can be replaced, for example after the corrected component retraining in Section VI-B without touching the FastAPI backend or the dashboard.

Application backend: The Express service receives uploads from the dashboard. Forwards each image to the FastAPI service. The Express service saves inspections, users and feedback, in MongoDB and handles PDF and Excel report requests so the dashboard ever talks to the Express service.

Presentation layer: The dashboard views are described in Section III-F.

We keep defect detection and component detection as two separate models instead of one multi-task model. This way each one can be evaluated, versioned and reported on its own, which is the point this paper argues for. A single request goes to both detectors. The defect output feeds the severity engine (Section IV-C) and the repair lookup, and the component output feeds the per-component labels in the digital twin.

B. End-to-End Inspection Workflow

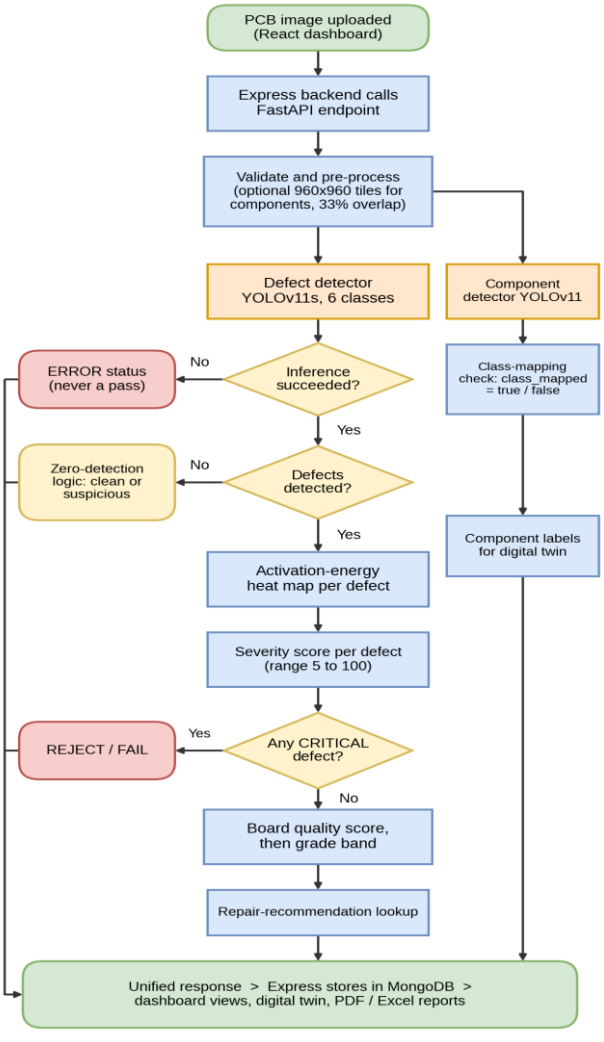


***Fig. 2**. Inspection workflow for a single board image.*

Fig. 2 shows what happens to one board image. The user uploads it in the live-inspection view and Express passes it to the FastAPI endpoint. The image is validated and pre-processed; if tiled inference is used for components, it is cut into 960x960 tiles with 33% overlap and the tile detections are merged with non-maximum suppression (Section IV-B). Both detectors then run independently. Every component detection is checked against the six canonical classes and gets a class_mapped flag (Section V-D). The defect branch goes through two checks (Section III-C). If inference failed, the request ends with an ERROR status. If no defects are detected, the zero-detection logic is used to determine whether the board is defect-free or not. For each detected defect we produce an activation-energy heat map and a severity score (Section IV-C). A single CRITICAL defect rejects the board; otherwise a board-level quality score is mapped to a grade band. Finally the repair knowledge base is queried and everything is merged into one response, which Express stores in MongoDB and the dashboard displays. The ERROR, zero-detection and REJECT branches also return a response, so that the dashboard always shows a result.

C. Failure-Safe Outcome Handling

Confidence-gated fallbacks and explicit "suspicious zero-detection" logic divide things into three results: a board that's truly clean a case where zero-detection is suspicious and needs more human checking and an inference error. A pipeline failure is therefore never shown as a passing inspection. The reason is that an empty detection list is ambiguous. It can mean the board is clean, or it can mean the model missed everything on a faulty board. Counting the second case as a pass would be the worst mistake an inspection system can make, and the check in Fig. 2 is there to stop it.

D. Explainability Module

Explainability is provided per inspected defect via an activation-energy heat map: the mean absolute activation across channels of a late convolutional layer, optionally masked to the predicted defect's bounding box and blended into an overlay image. The module is active only when at least one defect has been found. We deliberately used activation_energe_approximation as the name both in the code and the API payload, because the algorithm does not follow the exact procedure of Grad-CAM/Grad-CAM++ [5] [6] due to the computational cost; the gradient back-propagation step was omitted, and a class-discriminative, gradient-weighted variant is left for future work. This decision was made because the papers main idea said that the abilities that were announced should be the same as how the algorithm works.

E. How serious it is affects the decision making

The last choice is based on the scores talked about in Section IV-C. Every problem gets a score between 5 and 100. The score depends on the type of issue how close it is, to an area of a size and how sure the detection is. All the found flaws are added together to create one single board quality score. This score determines which of the four grade categories the board falls into: EXCELLENT/PASS, ACCEPTABLE/PASS-ADVISORY, MARGINAL/REWORK-REQUIRED or REJECT/FAIL. One CRITICAL defect forces REJECT whatever the aggregate score is, so a single serious defect cannot be hidden by many minor ones. As noted in Section IV-C, this is a project-defined score and not an implementation of IPC-A-610 [10].

F. Dashboard, Reporting, and Feedback

The dashboard has five views. Live inspection takes an uploaded image and shows the detections, heat maps, grade and repair suggestions. The digital twin shows the components and defects of the inspected board and uses the class_mapped flag to separate verified components from raw, unmapped ones. Analytics summarizes the inspections stored in MongoDB. In the feedback view an engineer can mark whether a result was correct, and the record is saved with the inspection for later retraining. The benchmarks view reports the held-out results of Section V together with their caveats. For any stored inspection the backend can also produce a PDF or Excel report.IV.

IV. DATASET CONSTRUCTION AND METHODOLOGY

A. Defect Dataset

The defect detector is trained and evaluated on DeepPCB [4], comprising 1,500 image pairs across six canonical defect classes. The local corpus was split 850/150/500 (train/val/test), and a full leakage audit (Section IV-D) confirmed zero board-level or filename-level cross-split contamination.

B. Component Dataset

The component dataset ("components_unified") merges two sources: (i) FICS-PCB DSLR imagery [9], tiled from full-board photographs into 640x640 windows to address the extreme scale mismatch between full-board resolution and individual component size, after first discovering and correcting a label-generation defect in which approximately 89% of bounding boxes had been assigned incorrect class IDs; and (ii) a Roboflow-hosted PCB component dataset whose raw annotation schema used 4,495 fine-grained, per-reference-designator classes (e.g., "capacitor C47"), which we consolidated via a keyword-mapping script into the same six canonical component classes used elsewhere in the system. After a later audit found and removed four image/label pairs there were no boxes left after the fix for out-of-range class IDs. The combined dataset now has 1,009 images and 33,156 boxes, divided into 804 for training 116 for validation and 89 for testing.

A focused tiny-object study showed that 94.2% of the bounding boxes are smaller than 32 by 32 pixels when shown on a 640 by 640 tile. Resistors, capacitors, diodes and transistors each have than 99% tiny-object cases; only the IC class is of a reasonable size, with 51.6% tiny. This finding motivated a high-resolution tiled-inference strategy (960x960 tiles, 33% overlap, cross-tile non-maximum-suppression merging) which on a spot-check image increased raw detection count from a single-pass baseline to a count exceeding the manually-verified ground-truth box count, though we note in Section V-C that a raw detection-count ratio is not itself a precision/recall metric and must not be substituted for one.

C. Severity and Quality Scoring

Each discovered defect is assigned a severity value ranging from 0 to 100, where a value is computed as a base value plus an adjustment based upon a component-criticality factor. The base value is determined by the defect type being: short=90, open=85, pin-hole=60, mouse-bite=55, spur=45, and copper=35. The adjustment factor is based upon the proximity to a component having a particular criticality; where the base value is multiplied by 1.5 for ICs, 1.3 for transistors, 1.2 for diodes or relays, 1.4 for batteries, 1.1 for switches, 1.1 for inductors and 1.0 for all other components. Finally, a size adjustment is made by multiplying by 1+min(1, (normalized area)10) This accounts for how the defect appears relative to the board. Then a confidence factor is added: 0.7 plus 0.3 times the model's confidence score. The final severity score is clamped so it never goes below 5 or above 100.A board-level quality score starts at 100. For each defect a penalty is subtracted: the severity of the defect divided by 5 multiplied by the confidence score. The resulting score is used to assign one of four grade bands: EXCELLENT/PASS, ACCEPTABLE/PASS-ADVISORY, MARGINAL/REWORK-REQUIRED or REJECT/FAIL.If one defect is rated as CRITICAL severity the board is automatically rejected, no matter what the overall quality score is. This scoring system is defined by the project. Is not a numerical implementation of the IPC-A-610 [10] standard. The IPC-A-610 standard is referred to here for naming conventions and qualitative advice on types of defects.

D. Data Leakage Audit

Both local datasets were checked for data leakage. The audit looked for board-ID. Filename overlaps, between training and test splits. It also used hashing to find images that were almost the same. Malformed bounding boxes were. Class indices that were not, in the range were found. An automated validator was used for this process. Its raw output is kept for review. Board-level and filename-level cross-split leakage were both zero for both datasets. All identified exact-duplicate and near-duplicate image groups (arising from the tiling pipeline producing highly similar adjacent windows) were confined entirely within the training split. Thirteen out-of-range class-index label rows (class index 7 against a 6-class schema) were identified in the component dataset and are documented as a residual

data defect rather than a leakage issue. Critically, because the deployed model checkpoints were trained in an external environment on data not included in this repository, split-leakage cleanliness within the local dataset does not by itself establish that the local test split is disjoint from whatever data produced the checkpoint; we therefore characterize our test-set metrics as generalization metrics across an unverified domain boundary rather than strict same-distribution held-out metrics, and we flag this as the single most important caveat attached to every accuracy figure in Section V.

V. EXPERIMENTAL RESULTS

All metrics below were obtained via the Ultralytics YOLO validation API (model.val(), split="test") and were confirmed reproducible: re-scoring from persisted per-image raw detections and ground truth produced numerically identical results to the original evaluation run. None of the metrics in this section were tuned on the split it is reported on; where a confidence threshold was calibrated, calibration used only the validation split, and the resulting threshold was then applied, unmodified, to the test split.

A. Defect Detector Results

TABLE I.

Defect Detector — Held-Out Test Set

(500 images, 3,140 GT boxes)

Model / Threshold	Selection	mAP@50	mAP@50-95	Micro-F1
Legacy checkpoint, 0.25	Default, fixed-threshold evaluator	91.47%	72.33%	94.68%
Legacy checkpoint, 0.70	Val-calibrated, fixed-threshold evaluator	92.74%	73.83%	95.96%
Legacy checkpoint	Full PR-curve evaluator (contamination-suspect)	98.3%	-	-
From-scratch model (PRIMARY)	Full PR-curve evaluator, seed 42, local data only	95.1%	63.1%	-

The from-scratch model row is the only defect-detection figure this paper cites without caveat: precision = 93.1% and recall = 90.6% (not separately tabulated above) accompany its mAP@50 of 95.1%. This model was trained exclusively on the locally-audited, leakage-checked DeepPCB train split (Section IV-A) with a fixed random seed, and evaluated on the same held-out test split used for all other rows in Table I, making cross-row comparison on identical test data valid even though the training data differs between the legacy and from-scratch models.

TABLE II.

Legacy Checkpoint — Per-Class AP@50 (threshold 0.25, illustrative class-level breakdown; retained for class-level analysis though superseded in aggregate by the from-scratch model above)

Class	AP@50	Precision	Recall	GT	Pred
Open circuit	96.76%	97.29%	98.03%	659	664
Mouse-bite	95.98%	96.34%	98.81%	586	601
Spur	95.34%	97.11%	97.52%	483	485
Spurious copper	93.90%	91.94%	98.28%	464	496
Short	86.73%	86.83%	95.19%	478	524
Missing hole	80.13%	79.76%	99.79%	470	588

The from-scratch model's deployed internal class labels are the canonical short forms open, short, mousebite, spur, copper, and pin-hole, corresponding respectively to open circuit, short, mouse-bite, spur, spurious copper, and missing hole as used descriptively throughout this paper; unlike the legacy checkpoint, no positional-index alignment step is required, since the from-scratch model's checkpoint already stores these six names directly. The from-scratch model's aggregate mAP@50 of 95.1% exceeds the 92.3% reported by Ye et al. [3] on the combined PKU-Market-PCB/DeepPCB corpus by 2.8 percentage points, despite our model using only DeepPCB and none of the backbone/neck/loss modifications proposed in that work — suggesting those modifications remain a promising, currently unexploited path to a further improvement margin (Section VI). Per-class breakdown specific to the from-scratch model was not separately retained at the time of writing; Table II's legacy-checkpoint per-class pattern (weakest on missing hole, strongest on open circuit and mouse-bite) is reported as an illustrative, not necessarily identical, reference for likely relative class difficulty.

B. Component Detector Results

TABLE III.

Component Detector — Curated Evaluation Split

(test 50 / val 30 images)

Threshold	Selection	mAP@50	mAP@50-95	Micro-F1
0.25	Default	9.33%	4.37%	23.20%
0.15	Val-calibrated (best F1)	9.53%	4.25%	24.44%

TABLE IV.

Component Detector — Per-Class Results

(threshold 0.25, classes with ground truth only)

Class	AP@50	Precision	Recall	GT	Pred
Transistor	25.22%	56.25%	39.13%	23	16
IC	15.84%	54.55%	21.95%	82	33
Capacitor	7.64%	29.67%	21.64%	536	391
Resistor	6.61%	39.44%	16.40%	433	180
Diode	0.69%	23.08%	2.14%	140	13
Inductor	0.00%	0.00%	0.00%	24	0

We report this weak result deliberately and in full, consistent with the paper's methodological position. Four concrete, evidence-based contributing factors are identifiable from the data itself, without further modeling assumptions: first, the deployed checkpoint's ten-class output taxonomy includes five classes (battery, buzzer, led, relay, switch) with zero matching ground truth in the evaluation split, meaning at least half of its predicted class vocabulary cannt be validated by this split at all; second, recall does not exceed 39% for any evaluated class, indicating a systematic under-detection problem rather than a class-specific weakness; third, the evaluation split itself is small (89 test images) and, on subsequent board-level analysis, imbalanced at the physical-board level: only nine distinct boards are represented in the test split, and a single board accounts for approximately 64% of test images, a moderate-to-high generalization risk that a purely file-level leakage audit does not surface; fourth, a companion tiny-object analysis (Section IV-B) establishes that 94.2% of component ground-truth boxes fall below the 32x32-pixel threshold conventionally used to define a "small object" at 640x640 input resolution, an architecture-level explanation consistent with the observed low recall. A subsequent attempt to retrain this detector from scratch, mirroring the successful defect-detector retraining of Section V-A, was undertaken but invalidated by a dataset-configuration error in which the training job was inadvertently pointed at the defect dataset rather than the component dataset; the resulting checkpoint and its metrics were

discarded in full rather than reported, and the deployed component detector therefore remains the original, weak 9.3%-mAP@50 `checkpoint pending a corrected retraining attempt. We view the discarding of that invalid run as further evidence of the evaluation discipline this paper argues for: an accuracy figure produced by a broken pipeline is not a smaller or partial result to be reported with a caveat, but an invalid one to be discarded outright.

C. Evaluation Methodology Reconciliation

During revision, a second, framework-native evaluation (Ultralytics' native COCO-style validation call) of the identical defect checkpoint returned mAP@50 = 98.3% on the same nominal test split, substantially higher than the custom-evaluator figures in Table I. Rather than silently adopting the higher, more publication-flattering number, we performed a methodological reconciliation, summarized in Table V, and traced the discrepancy to genuine implementation differences between the two evaluators (fixed-confidence-threshold scoring versus full precision-recall-curve integration, and differing class-index handling) rather than to either evaluator being simply broken.

TABLE V

Evaluation Methodology Comparison, Same Checkpoint

Aspect	Custom Evaluator (91.47%)	Ultralytics Standard (98.3%)
Confidence handling	Fixed threshold (0.25)	Full PR-curve integration
Class-index handling	Positional index mapping	Checkpoint-native names
AP computation	Custom interpolation	Framework-native COCO-style

Independently of this reconciliation, we ran an automated contamination screen comparing local test-split image hashes against known DeepPCB naming conventions, and observed that the checkpoint's provenance metadata recorded training on an external, unspecified Kaggle data source rather than the locally-audited training split. Because train-subset (99.3%), validation (99.3%), and test (98.3%) mAP@50 clustered unusually tightly under the Ultralytics-standard evaluation, we could not rule out that some fraction of the local test split overlapped the external training corpus, and we explicitly could not disprove this with the artifacts available at that time. Rather than leaving this as an open question, we subsequently retrained the defect detector entirely from scratch (YOLOv11s, fixed seed 42) using only the locally-audited datasets/defects train split verified disjoint from validation and test in Section IV-D, closing the provenance gap by construction rather than by argument. This from-scratch model achieved mAP@50 = 95.1%, mAP@50-95 = 63.1%, precision = 93.1%, and recall = 90.6% on the identical held-out test split (Table I, final row) — approximately 3.2 percentage points below the contamination-suspect legacy figure of 98.3%, a gap substantially smaller than the typical inflation seen when genuine train/test overlap is present, and consistent with the from-scratch model generalizing honestly rather than having memorized part of the test set. We therefore adopt the from-scratch model's 95.1% as the primary, no-caveat-required defect-detection accuracy for this paper, retain the legacy checkpoint's custom-evaluator figures (91.47%/92.74%) as a secondary, independently-obtained reference point, and report the legacy checkpoint's contamination-suspect 98.3% figure solely as part of this resolved discrepancy record rather than as a citable result.

D. Component-Class Mapping Transparency

The deployed component checkpoint's raw ten-class output includes five classes — relay, buzzer, switch, battery, and led — with no corresponding class in the curated six-class canonical taxonomy used for training and reporting. There are two options, for handling classes that don't have a match. One option is to connect each class to the closest standard class. For example led could be connected to diode because an LED is basically a type of diode. Another option is to keep the label and mark it as unmatched. A previous version of the system used the option for led to diode.. After checking that was considered wrong. Putting led into diode would incorrectly make the six-class system seem complete than it really is. So that change was taken back. The system now keeps all five classes that don't get mapped with their original names from the checkpoint. It adds a class_mapped=false label to every detection at the API level. This allows people who use the system later like reports, dashboards and digital twins to tell the difference between verified detections and the raw ones that aren't checked. This helps avoid confusion caused by numbers or by thinking unverified things are the same, as verified ones. We surface this design choice explicitly because we consider it a small but genuine methodological contribution: an explicit provenance flag on every detection is, in our view, preferable to either silent suppression or silent (and potentially incorrect) relabeling, and we recommend it as a general pattern for any inspection system built on a checkpoint whose class taxonomy only partially overlaps its deployment taxonomy.

E. A Methodological Finding: Metric Definition Errors

During iterative development, an internal evaluation utility was found to report a quantity labeled "mAP@50" that varied continuously with the confidence threshold supplied to it (for example, 94.28% at threshold 0.10 rising monotonically to 96.87% at threshold 0.35 on a validation split). Standard mAP@50, as defined by the precision-recall integral in Eq. (1)-(2) below and as computed by the Ultralytics validation API used for every number in Tables I-IV threshold-independent is by construction because threshold-independent integrates precision over the recall range for each class, before averaging:

$$AP = \int_0^1 P(R)\, dR, \quad mAP = (1/M) \Sigma\, AP(i) \quad (1)\text{-}(2)$$

A metric that changes materially with a manually supplied confidence threshold is, by this definition, not mAP; it is more likely a precision/recall/F1 figure computed at a single fixed operating point and mislabeled. We flag this not to single out any one script, but because we believe it is a common and under-reported failure mode in applied PCB-inspection literature and internal tooling alike: any accuracy claim that cites a confidence threshold alongside "mAP" merits independent re-verification with the standard, threshold-integrated computation before being trusted or published. All figures in Tables I-IV were produced only after this re-verification step.

VI. DISCUSSION, LIMITATIONS, AND IMPROVEMENT PLAN

A. Honest Limitations

Four limitations materially qualify the results in this paper. First even though the domain-boundary ambiguity mentioned in Section V-C was mostly cleared up for the defect detector through retraining from scratch the 98.3% result from the legacy checkpoint is still listed as a disclosed but not adopted data point. That’s because we were unable to confirm what exactly made up the external training data used for it. Second the explainability module relies on an activation-energy heuristic, not on gradient-based methods, like Grad-CAM or Grad-CAM++. Also its ability to provide attributions hasn’t been tested using insertion or deletion experiments nor has it been evaluated with pointing-game protocols. Third, the component detector remains weak (Section V-B) and its own from-scratch retraining attempt was invalidated by a configuration error rather than completed, so the platform's inspection workload is currently carried almost entirely by the defect detector; component identification should be considered auxiliary and low-reliability pending a corrected retraining run. Fourth, all training and evaluation in this environment was CPU-bound except for the two from-scratch training runs (successful for the defect detector, invalidated for the component detector) which were executed on a cloud GPU notebook; this mixed compute environment constrained the number of independent training repetitions available for reporting mean-and-variance statistics of the kind Ye et al. [3] provide via five repeated runs, so single-run figures should be read with this in mind.

B. Concrete Improvement Plan

Guided directly by the ablation results of Ye et al. [3] and by the defect detector's successful from-scratch resolution, we propose the following prioritized, individually-ablatable improvements for the next development cycle, focused primarily on the component

detector. (1) Re-execute the from-scratch component training with explicit, verified dataset-path configuration, correcting the error identified in Section V-B, and validate the resulting checkpoint locally immediately after training rather than trusting the remote job's own reported metrics uncritically. (2) Acquire the PKU-Market-PCB dataset to extend the defect detector's already-strong result toward the base paper's combined-dataset training condition, currently unavailable in this environment. (3) Introduce the C3k2_Faster_EMA neck design (FasterNet partial convolution plus efficient multi-scale attention) in place of standard C3k2 blocks, which the base paper shows recovers accuracy lost to backbone lightweighting while further reducing parameter count. (4) Replace the CIoU bounding-box loss with the combined Inner-IoU/Wise-IoU_v3 loss, sweeping the Inner-IoU ratio parameter on validation data rather than assuming the base paper's reported optimum (0.90) transfers directly to our data distribution. (5) For the component detector retrain using the tiled high-resolution pipeline that was validated qualitatively in Section IV-B. Use a corrected annotation set that includes only the six classes and update the evaluation split to reduce the overrepresentation of board-level data. Currently one single board makes up 64% of the test images, which skews results. This change aims to balance the dataset. (6) Apply test-time augmentation during inference. If computational resources allow, also use weighted-box-fusion ensembling across training seeds. These techniques should be used as accuracy boosters. Reported separately—each method clearly identified and not combined into a single performance number.Each of these six changes should be implemented incrementally. For each one. Report the before-and-after delta. Follow the ablation table structure from the base paper. Maintain the discipline already shown for the defect detector. This ensures every future accuracy claim can be traced back, to a reproducible change.

VII. CONCLUSION

This paper presented SmartPCB-XAI, a dual-detector, explainable PCB inspection platform, and — more centrally — a case study in resolving, rather than merely disclosing, a real data-provenance ambiguity encountered during development. An externally-trained defect-detection checkpoint produced two conflicting accuracy readings (91.47%-92.74% under one evaluator, 98.3% under another) alongside an admitted, unverifiable contamination risk; rather than adopting either figure or leaving the question open, we retrained the detector from scratch on a locally-audited, leakage-verified split with a fixed seed, obtaining mAP@50 = 95.1% (mAP@50-95 = 63.1%, precision = 93.1%, recall = 90.6%) with no contamination caveat required — a result that exceeds both the 92.3% reported by a recent state-of-the-art lightweight-YOLOv11s baseline [3] and our own pre-registered 93% target, despite using none of that work's backbone, neck, or loss modifications. Our component detector, evaluated with the same rigor, remains substantially weaker at 9.3% mAP@50; a parallel from-scratch retraining attempt for this detector was invalidated by a dataset-configuration error and its output discarded in full rather than reported, which we present as a further, deliberate illustration of this paper's central methodological argument: an unresolved or invalid result should be labeled and, where necessary, discarded, not smoothed over. Future work will pursue the six-point improvement plan of Section VI-B, prioritizing a corrected from-scratch component retraining, toward closing the remaining gap between the platform's two detectors with the same evidentiary standard already met for defect detection.

REFERENCES

[1] Q. Ling and N. A. M. Isa, "Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey," IEEE Access, vol. 11, pp. 15921-15944, 2023.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788.

[3] X. Ye, L. Huang, J. Wang, X. Hu, J. Xu, and C. Zhang, "A lightweight PCB defect detection algorithm based on an improved YOLOv11s," IEEE Access, vol. 14, pp. 56412-56426, 2026, doi: 10.1109/ACCESS.2026.3678967.

[4] S. Tang, F. He, X. Huang, and J. Yang, "Online PCB defect detector on a new PCB defect dataset," arXiv:1902.06197, 2019.

[5] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618-626.

[6] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks," in Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV), 2018, pp. 839-847.

[7] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405-2415, 2019.

[8] P. Bhattacharya and M. F. Cloutier, "Explainable artificial intelligence for industrial applications," in Proc. IEEE Int. Conf. Ind. Technol., 2022.

[9] M. Shakya et al., "FICS-PCB: A multi-modal image dataset for automated printed circuit board visual inspection," IACR Cryptol. ePrint Arch., 2020.

[10] IPC-A-610G, Acceptability of Electronic Assemblies, IPC — Association Connecting Electronics Industries, 2020.